

5-Fold Cross-Validation (track-level) - Notes

This notebook performs 5-fold cross-validation. Important implementation detail: folds are created at the *track* level using GroupKFold/GroupShuffleSplit so that no slices from the same original audio track appear in both training and validation for any fold.

Workflow (recommended):

1. Start from the list of original track files and their genre labels.
2. Create 5 stratified folds by genre using `GroupKFold` with `groups=track_id`.
3. For each fold: take training tracks and generate 3-second non-overlapping slices for model training; generate slices for validation tracks for evaluation. Do NOT slice first and then split by slice.

Below is a short template showing how to create the folds and then slice tracks after the fold is decided.

01c — 5-Fold Cross-Validation (UNet_Audio_Classifier) on GTZAN

Pipeline per WP4 (opzionale): esegue K=5 fold sul solo UNet, riportando media±std.

```
In [6]: import os, pickle, numpy as np, pandas as pd, tensorflow as tf, keras, time,
from keras import layers, models, callbacks
from sklearn.model_selection import StratifiedKFold
from pathlib import Path

# -----
# GPU memory growth + Mixed Precision for lower memory footprint
# -----
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
            print(f"GPU(s) detected: {[tf.config.experimental.get_device_details(g) for g in gpus]}")
    except Exception as e:
        print("Warning: could not set memory growth:", e)
try:
    policy = keras.mixed_precision.Policy('mixed_float16')
    keras.mixed_precision.set_global_policy(policy)
    print("Mixed precision enabled:", keras.mixed_precision.global_policy().name)
except Exception as e:
    print("Warning: could not enable mixed precision:", e)

# -----
# Paths and data loading
```

```

# -----
PROJECT_ROOT = Path(os.getcwd()).resolve().parent.parent
PROCESSED = PROJECT_ROOT/'data'/'processed'
REPORTS = PROJECT_ROOT/'reports'
REPORTS.mkdir(exist_ok=True)

X = np.load(PROCESSED/'X_train.npy'); y = np.load(PROCESSED/'y_train.npy')
Xv = np.load(PROCESSED/'X_val.npy'); yv = np.load(PROCESSED/'y_val.npy')

# Merge train+val to perform CV over the 80% pool
X_all = np.concatenate([X, Xv], axis=0)
y_all = np.concatenate([y, yv], axis=0)

# Optional: downcast inputs to float16 to reduce host memory (safe with mixed precision)
DOWNCAST_FP16 = True
if DOWNCAST_FP16 and X_all.dtype != np.float16:
    X_all = X_all.astype(np.float16)

y_all = y_all.astype(np.int32)
num_classes = int(len(np.unique(y_all)))
print("Data pool for CV:", X_all.shape, "| classes:", num_classes)

# -----
# Model definition (kept consistent with project; final Dense in float32)
# -----
def build_unet(input_shape, num_classes):
    i = layers.Input(shape=input_shape)
    x = layers.Conv2D(32,3,padding='same',use_bias=False)(i); x = layers.BatchNormalization(x)
    x = layers.Conv2D(32,3,padding='same',use_bias=False)(x); x = layers.BatchNormalization(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(64,3,padding='same',use_bias=False)(x); x = layers.BatchNormalization(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(128,3,padding='same',use_bias=False)(x); x = layers.BatchNormalization(x)
    x = layers.GlobalAveragePooling2D()(x); x = layers.Dropout(0.5)(x)
    o = layers.Dense(num_classes, activation='softmax', dtype='float32')(x)
    return models.Model(i,o)

# -----
# Training configuration and fold runner
# -----
BATCH_SIZE = 16 # lower batch to prevent OOM; reduce to 8 if needed
EPOCHS = 60
PATIENCE = 10
AUTOTUNE = tf.data.AUTOTUNE

def run_fold(train_idx, val_idx):
    # Ensure a clean graph for each fold
    keras.backend.clear_session()

    model = build_unet(X_all.shape[1:], num_classes)
    model.compile(optimizer=keras.optimizers.Adam(1e-3),
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    # tf.data pipelines to reduce memory pressure
    ds_train = (tf.data.Dataset.from_tensor_slices((X_all[train_idx], y_all[

```

```

        .shuffle(min(10000, train_idx.shape[0]))
        .batch(BATCH_SIZE)
        .prefetch(AUTOTUNE))
ds_val = (tf.data.Dataset.from_tensor_slices((X_all[val_idx], y_all[val_idx])
        .batch(BATCH_SIZE)
        .prefetch(AUTOTUNE))

cb=[callbacks.EarlyStopping(monitor='val_accuracy', patience=PATIENCE, restore_best_weights=True),
    callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=10, verbose=0, mode='min')]

h = model.fit(ds_train, validation_data=ds_val, epochs=EPOCHS, verbose=0)
best_va = float(np.max(h.history.get('val_accuracy', [0])))

# Cleanup to free VRAM/RAM
del ds_train, ds_val, h, model
keras.backend.clear_session()
gc.collect()

return best_va

# -----
# Cross-validation loop
# -----
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = []
fold_rows = []
print("Starting 5-fold CV...")
for k, (tr, va) in enumerate(skf.split(X_all, y_all), 1):
    t0 = time.time()
    print(f"Fold {k}/5...")
    acc = run_fold(tr, va)
    dt = time.time() - t0
    print(f"Fold {k} done: val_acc={acc:.4f}, time={dt:.1f}s")
    scores.append(acc)
    fold_rows.append({"Fold": k, "Val_Accuracy": acc, "Time_s": dt})

mean = float(np.mean(scores)); std = float(np.std(scores))
print('CV 5-fold Val Accuracy mean±std:', mean, std)

# Save summaries (per-fold and aggregate)
fold_df = pd.DataFrame(fold_rows)
fold_df.to_csv(REPORTS/'kfold_cv_unet_gtzan_folds.csv', index=False)
pd.DataFrame([{'Model': 'UNet_Audio_Classifier', 'Dataset': 'GTZAN', 'K': 5, 'Val_Acc': mean, 'Val_Std': std}]).to_csv(SUMMARY/'summary.csv', index=False)
print('Saved per-fold:', REPORTS/'kfold_cv_unet_gtzan_folds.csv')
print('Saved summary:', SUMMARY/'summary.csv')

# Display a neat summary table
try:
    from IPython.display import display
    display(fold_df)
except:
    print(fold_df)

```

Warning: could not set memory growth: Physical devices cannot be modified after being initialized
Mixed precision enabled: mixed_float16
Data pool for CV: (8000, 128, 128, 1) | classes: 10
Starting 5-fold CV...
Fold 1/5...
Data pool for CV: (8000, 128, 128, 1) | classes: 10
Starting 5-fold CV...
Fold 1/5...
Fold 1 done: val_acc=0.9006, time=152.9s
Fold 2/5...
Fold 1 done: val_acc=0.9006, time=152.9s
Fold 2/5...
Fold 2 done: val_acc=0.8944, time=145.7s
Fold 3/5...
Fold 2 done: val_acc=0.8944, time=145.7s
Fold 3/5...
Fold 3 done: val_acc=0.9144, time=147.9s
Fold 4/5...
Fold 3 done: val_acc=0.9144, time=147.9s
Fold 4/5...
Fold 4 done: val_acc=0.9038, time=174.2s
Fold 5/5...
Fold 4 done: val_acc=0.9038, time=174.2s
Fold 5/5...
Fold 5 done: val_acc=0.9087, time=122.1s
CV 5-fold Val Accuracy mean±std: 0.9043750047683716 0.006835106673852528
Saved per-fold: /home/alepot55/Desktop/projects/naml_project/reports/kfold_cv_unet_gtzan_folds.csv
Saved summary: /home/alepot55/Desktop/projects/naml_project/reports/kfold_cv_unet_gtzan.csv

	Fold	Val_Accuracy	Time_s
0	1	0.900625	152.923059
1	2	0.894375	145.697510
2	3	0.914375	147.913864
3	4	0.903750	174.203336
4	5	0.908750	122.072955